

History of Programming Languages

From Wires and Switches to High-Level Languages



Explore the fascinating evolution of programming through the decades



Early Computers and Physical Programming

"Early computers, like the Colossus, were programmed without a stored program by altering their circuitry or configuring physical controllers."



No Memory for Programs

In the very beginning, computers didn't have a memory to store programs.



Physical Changes

To make them do something, engineers had to physically change the wiring or flip switches on the computer.



Example: The Colossus

Used in WWII to decode messages, was programmed by connecting wires and setting switches to make it perform tasks.



Think of it like:

Teaching a robot to dance by manually moving its arms each time—very slow and inflexible.

Machine Language

"Later, programs could be written in machine language, where the programmer puts each command in a numeric format that the hardware can directly execute."



Numbers Only

After wiring, the next step was machine language - just numbers (0s and 1s or decimal numbers).



Direct Execution

This is the lowest-level language computers understand - each number corresponds to a specific action.



Example

If 101 means "add two numbers," the programmer has to write hundreds of

numbers to make even a simple calculation.

Input Methods

"These programs were read in decimal or binary form from various sources, such as punched cards, paper tape, magnetic tape, or switches on the computer's front panel."



Punched Cards

Cards with holes representing numbers



Paper Tape

Long strips of paper with holes



Magnetic Tape

Early storage like a cassette tape



Switches on Panel

Manually flipping switches for input



Imagine:

Writing your essay on a stack of punched cards instead of typing—very slow!

First-Generation Programming Language (1GL)

"First-generation programming languages (1GL) were later coined to refer to machine languages."



Fast for Computers

Machine languages were fast for the computer to execute

Hard for Humans

Difficult for humans to write and read

Second-Generation Programming Language (2GL) – Assembly

"Second-generation programming languages (2GL) or assembly languages were created, which were tightly linked to the instruction set architecture of the individual machine. These languages made software more human-readable and reduced time-consuming and error-prone address computations."



Short Words Instead of Numbers

Instead of writing numbers, programmers could write short words or codes like ADD, SUB, MOV.



Hardware Dependent

Still depended on the computer's hardware, but made programming less error-prone and faster.



Example:

Writing ADD A, B instead of 101001 to add two numbers.

High-Level Programming Languages (3GL)

"In the 1950s, the first high-level programming languages (3GL) were developed."



Human-Friendly

Closer to human language, easier to read and write

Focus on Logic

No need to think about switches or memory addresses
—just the logic of the problem

Early High-Level Languages

DE

Plankalkül

Developed by Konrad Zuse (1943–1945) for the Z3 computer in Germany.



Short Code

Suggested by John Mauchly in 1949, represented mathematical formulas in a readable format instead of machine code.



Translation Challenge:

"However, every time the program was performed, it had to be translated into machine code, which made the process significantly slower than executing the comparable machine code."

These early high-level languages were human-friendly, but computers couldn't run them directly. They needed a translation step (conversion into machine code), which made execution slower.

Autocode – First Compiled High-Level Language

"Alick Glennie created Autocode at the University of Manchester in the early 1950s. As a programming language, it employs a compiler to translate the language into machine code automatically."



Compiler Innovation



Historical Significance

Autocode was special because it used a compiler that automatically converts human-readable code into machine code once, so the program runs faster.

Created for Mark 1 computer in 1952, considered the first compiled high-level language.

Other Variants:

1

Mark 1 Autocode

R.A. Brooker, 1954

2

Mercury Autocode

For Ferranti Mercury computer

3

EDSAC 2 Autocode

1961

4

Atlas Autocode

For University of Manchester
Atlas 1

FORTRAN – First Widely Used High-Level Language

"In 1954, John Backus invented FORTRAN, the first widely used high-level general-purpose programming language with a functional implementation."



Formula Translation

FORTRAN = Formula Translation



Target Audience

Designed for scientists and engineers to write programs with math formulas



Still in Use

Still used today for supercomputers and high-performance computing



Examples of Use:



Weather Simulations



Nuclear Simulations



Space Programs

FLOW-MATIC – Early English-Like Language

"Grace Hopper designed FLOW-MATIC, an early programming language, for the UNIVAC I between 1955 and 1959."



English-Like Syntax

FLOW-MATIC allowed programmers to write instructions in English-like words



Business Applications

It inspired COBOL, the business programming language used in banks, insurance, and government



Timeline

Public release of FLOW-MATIC: 1958, prototype completed: 1955



Think of it like:

Writing a program that says "If customer balance > 0, approve loan" instead of using numbers or complex codes.

Summary of Early Programming Evolution

Generation	Language Type	Key Feature	Example
1GL	Machine Language	Numbers only	Binary code
2GL	Assembly	Short mnemonics	ADD, MOV
3GL	High-Level	Human-readable, compiler/interpreter	FORTRAN, Autocode, Short Code
English-like	FLOW-MATIC	Readable like English	Inspired COBOL



Story Version:

1

Step 1

Computers had no brains—they needed wires and switches.

2

Step 2

Then we used numbers (machine code).

3

Step 3

Assembly made it easier with small words.

4

Step 4

High-level languages made programming like writing math formulas.



5

Step 5

Compilers and English-like languages made programs fast and understandable.



Today:

Programming is easier, faster, and used everywhere—from apps on your phone to rockets in space!

Timeline of Programming History

1943-1944

Colossus

The first programmable electronic digital computer, used for codebreaking during

WWII. Programmed by physical switches and plugboards.

1952

Autocode

The first compiled high-level programming language, created by Alick Glennie for the Mark 1 computer at the University of Manchester.

1954

FORTRAN

The first widely used high-level general-purpose programming language, developed by John Backus at IBM. Still used today in scientific computing.

1955-1959

FLOW-MATIC

Developed by Grace Hopper for the UNIVAC I,

this was the first programming language to use English-like statements, directly inspiring COBOL.

1959

COBOL

Created by the Conference on Data Systems Languages (CODASYL), heavily influenced by FLOW-MATIC. Became the dominant business programming language for decades.